Project: StillNess — Wellness Booking System
Student: Pangilinan, Vein Carmell Golo
Submitted: April 6, 2026

1. GitHub Repository

Repository:	https://github.com/veinP/IT342-Pangilinan-Stillness
Branch:	main
Web folder:	web/

2. Final Commits

Hash	Commit Message	Scope
18b308b	IT342 Phase 3 – Web Main Feature Completed	Main Feature
93c6155	feat(web): Implement admin dashboard and management interface	Admin Panel
91e04cf	feat(web): Add base layout, routing, and authentication flow	Web Auth & Layout
a5e7dd1	feat(backend): Setup backend APIs, DTOs, and services for web integration	Backend

3. Screenshots

3.1 Main Feature Page / Screen

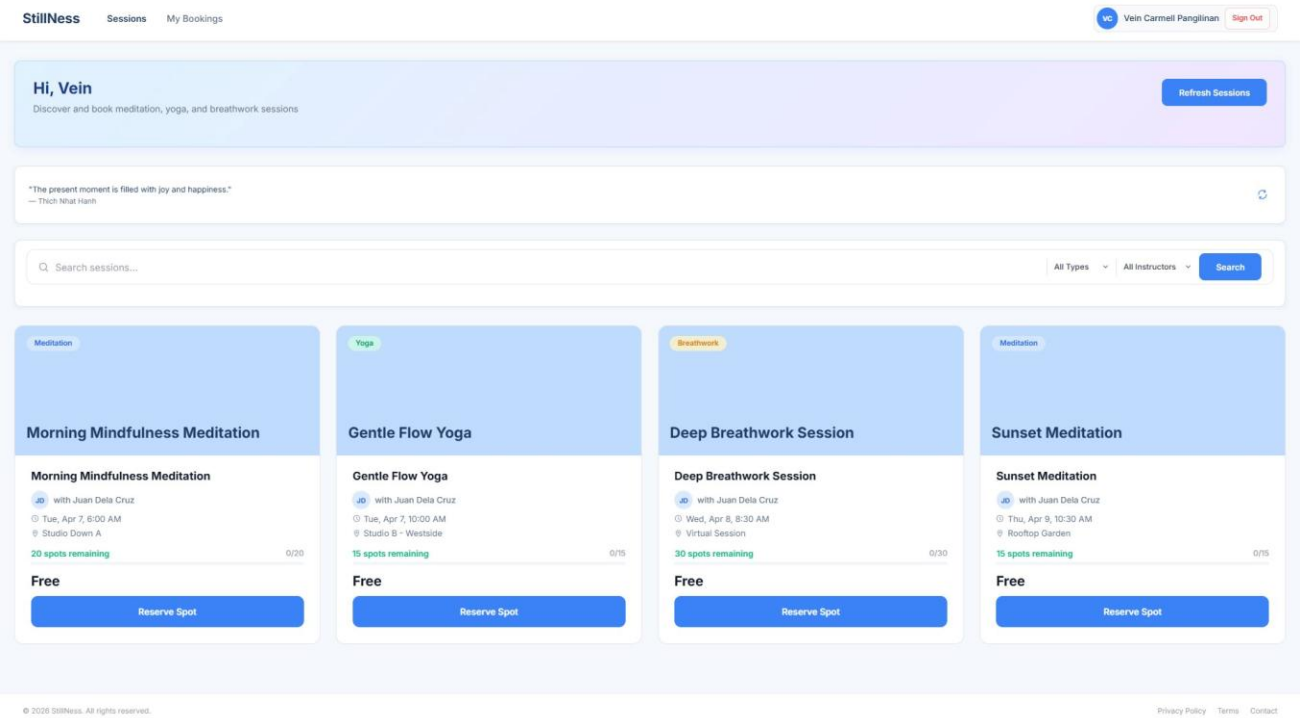

Figure 3.1 — Session Catalog Page

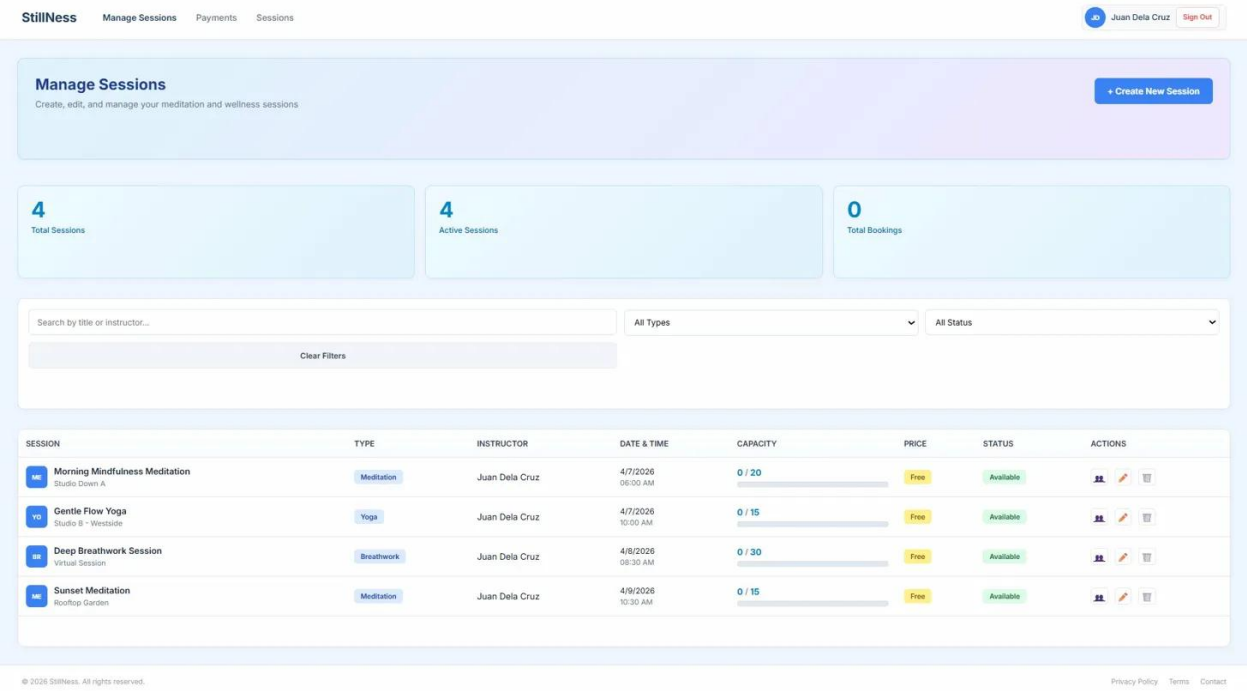

Figure 3.2 — Admin Manage Sessions

3.2 Adding or Using the Main Feature

Step 1 — Admin creates a new session:

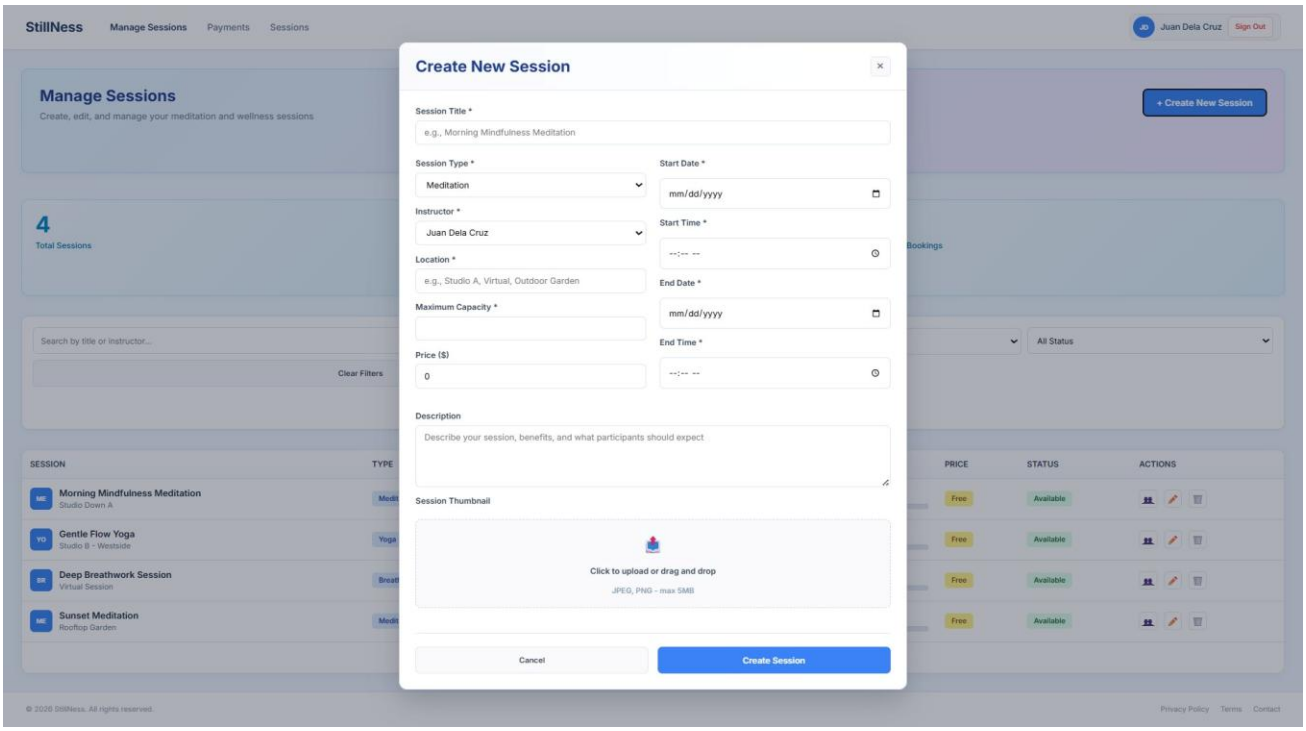

Figure 3.2a — Admin: Create New Session Modal — filling in title, type, instructor, location, capacity, and schedule

Step 2 — User views session details:

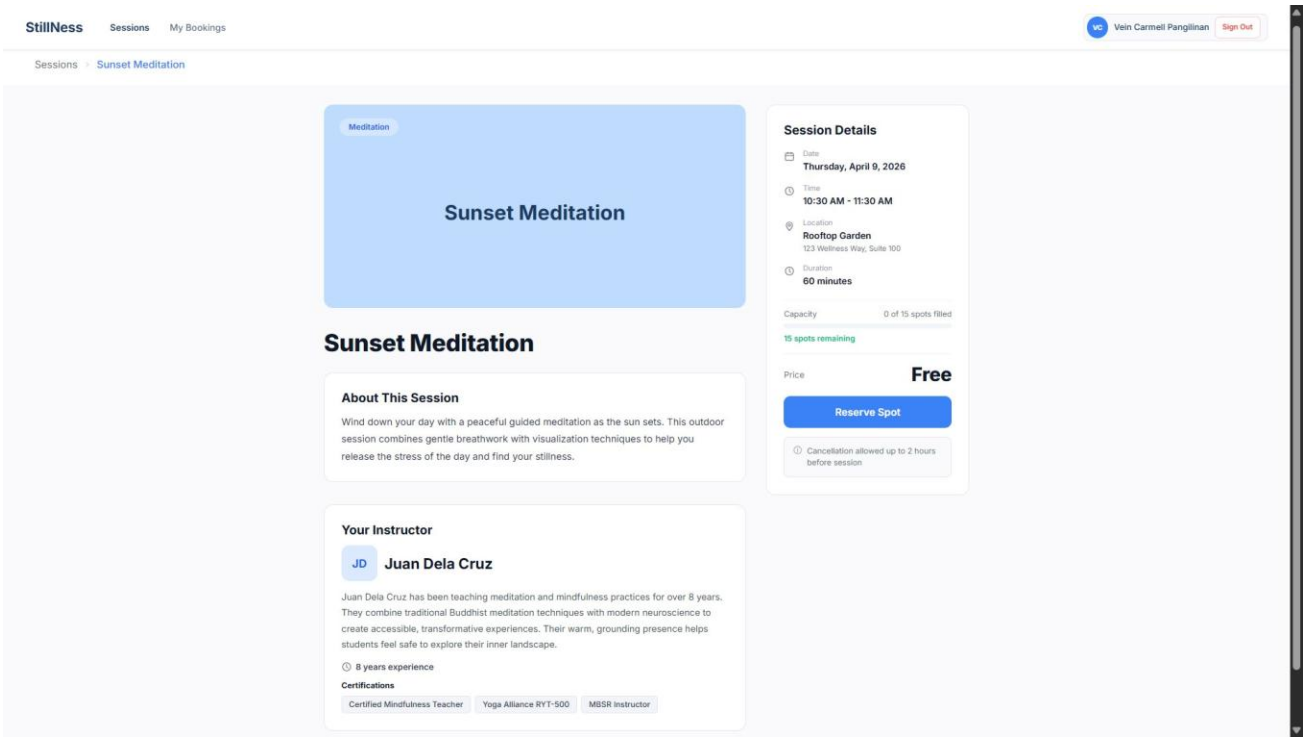

Figure 3.2b — Session Detail Page — Sunset Meditation with full info and Reserve Spot button

Step 3 — User proceeds to checkout:

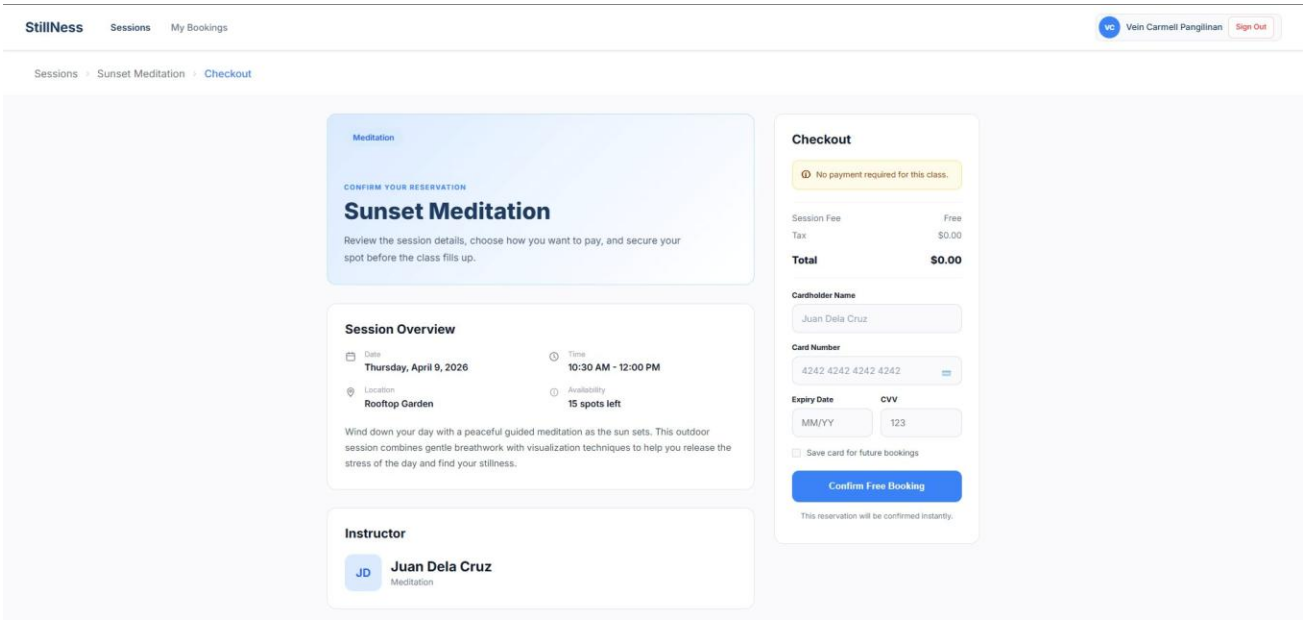

Figure 3.2c — Booking Checkout Page — session summary and payment confirmation form

3.3 Successful Output / Result

After confirming the booking, the user is redirected to My Bookings where the reservation appears with a Confirmed status badge.

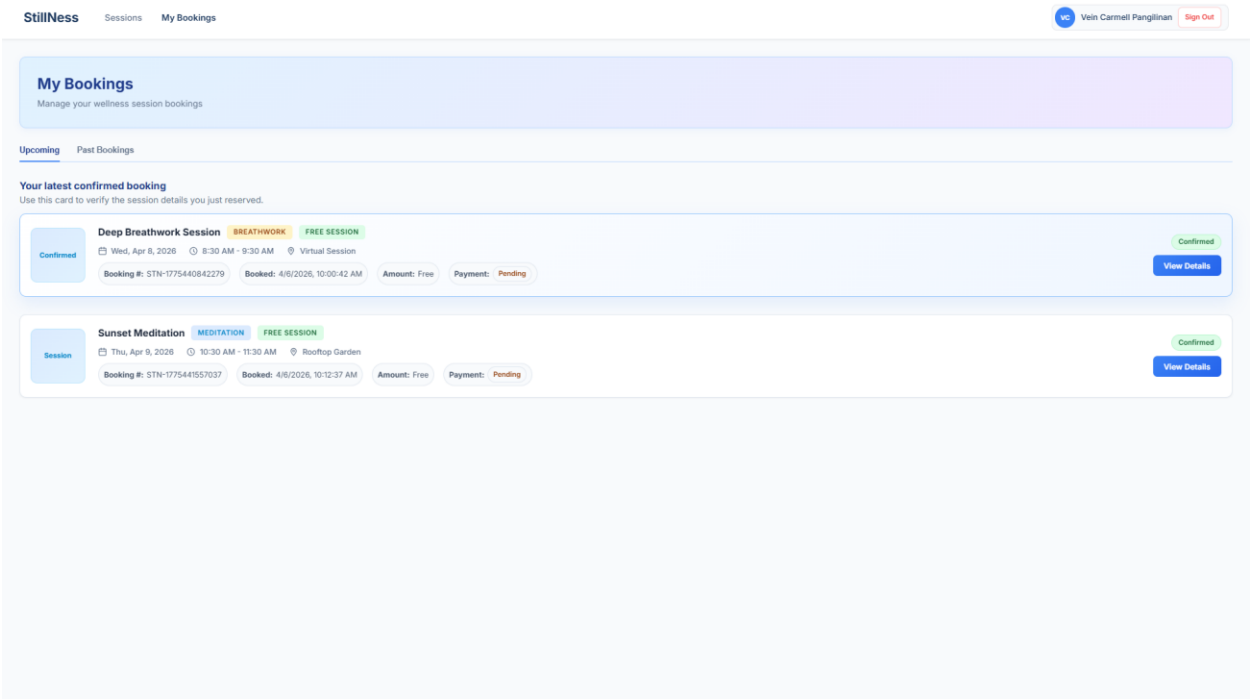

Figure 3.3a — My Bookings Page — confirmed booking

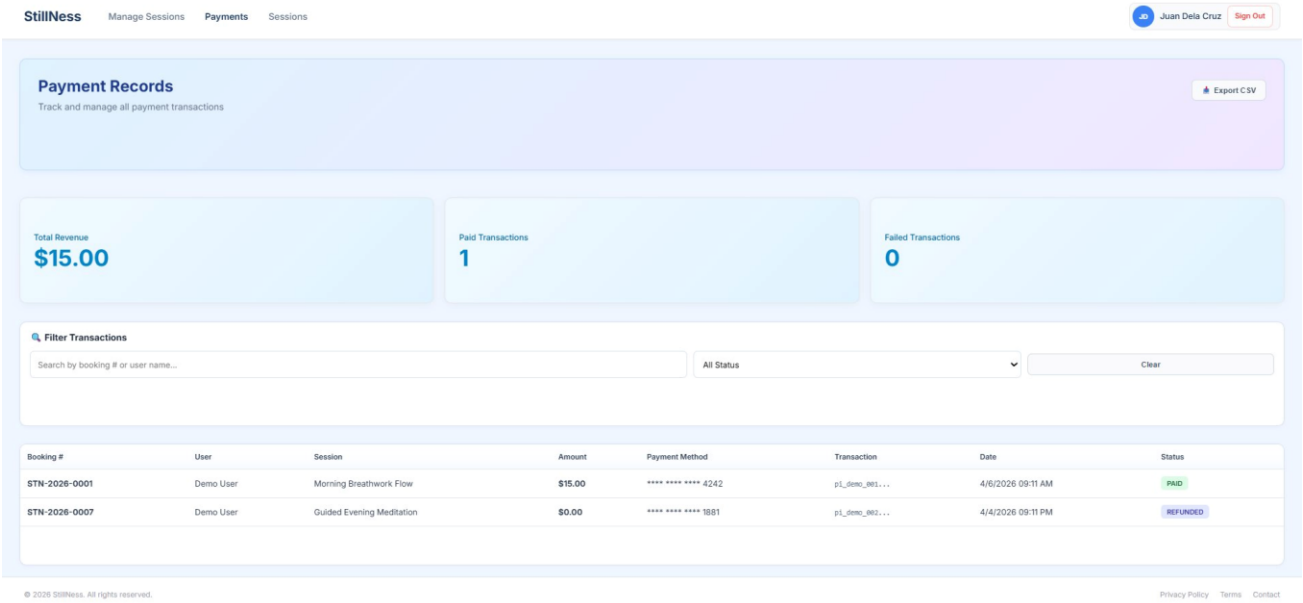

Figure 3.3b — Admin: Payment Records Page — recorded successfully

3.4 Database Record Related to the Feature

Database — sessions table in Supabase:

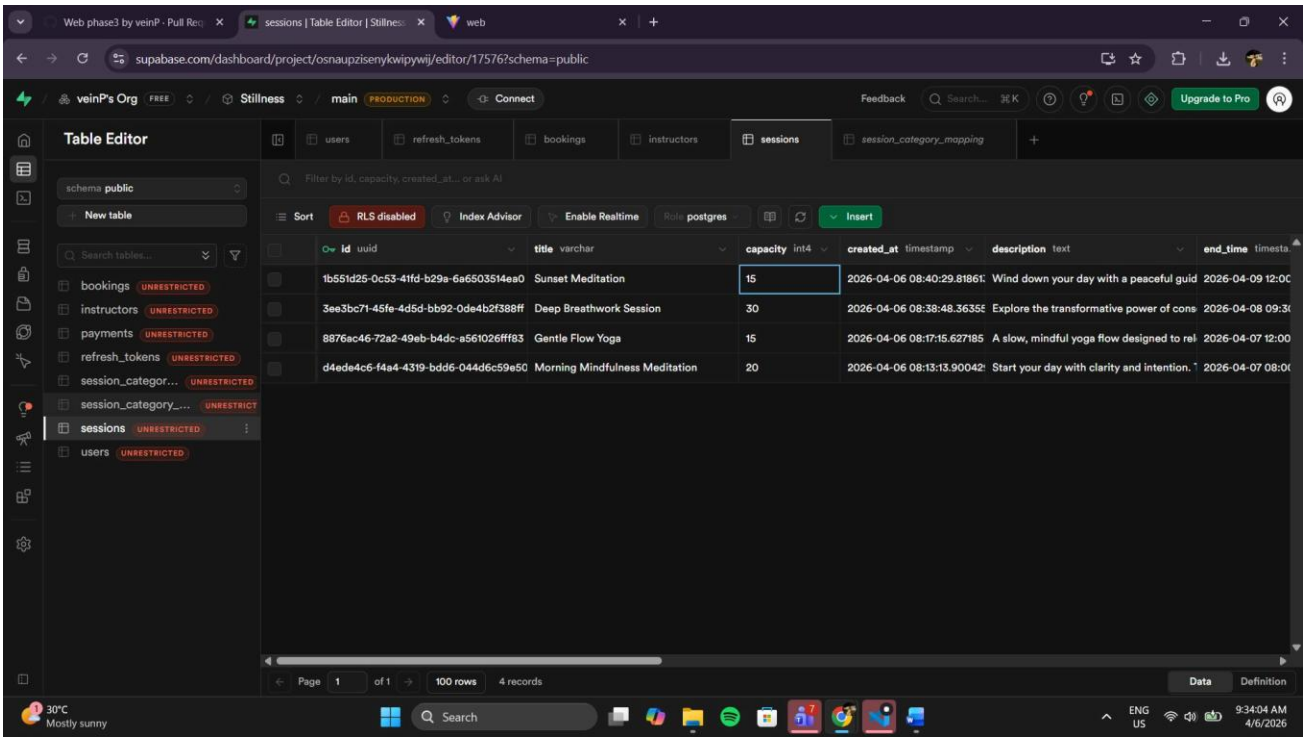

Figure 3.4a — Supabase — sessions table showing the created session record

Database — bookings table in Supabase:

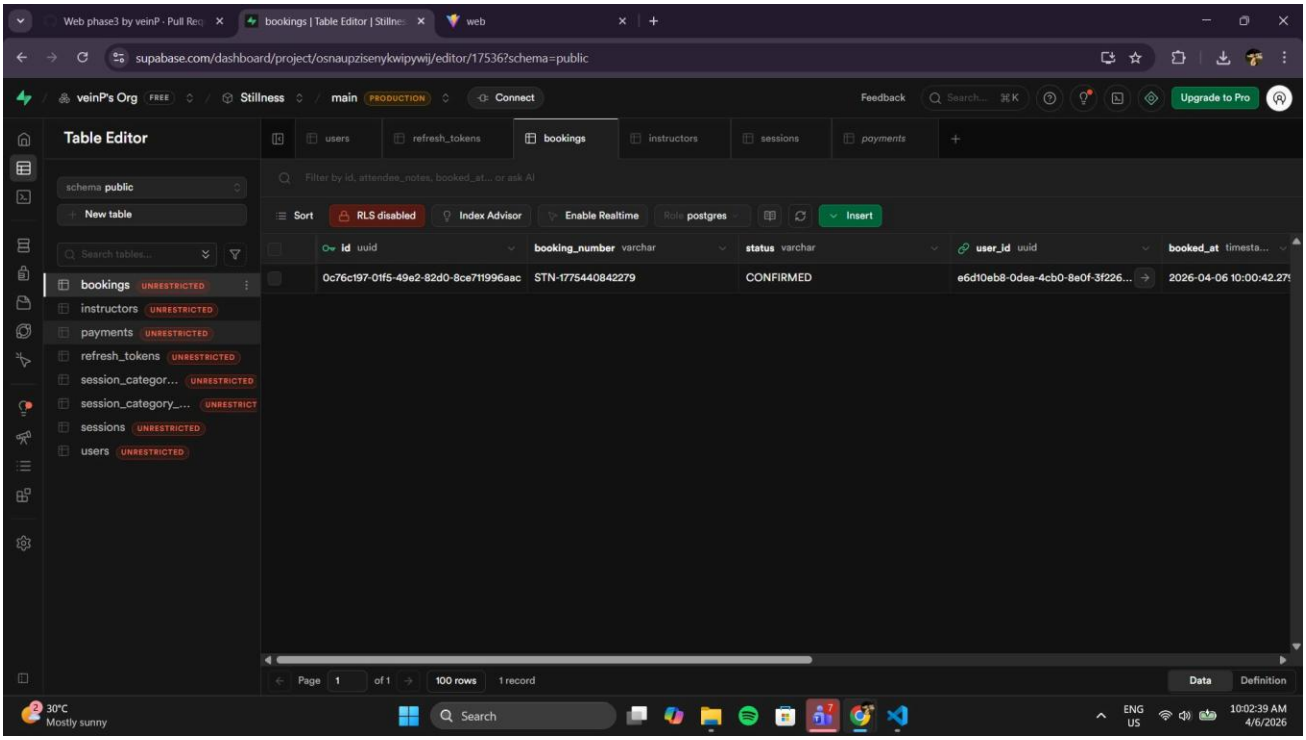

Figure 3.4b — Supabase — bookings table showing the confirmed booking record

4. Short Summary

4.1 Description of the Main Feature

The main feature of the StillNess web application is the Session Booking System. It enables authenticated users to browse available mindfulness sessions — including yoga, meditation, and breathwork — and reserve a spot directly through the web interface. The feature encompasses the full booking flow: session discovery, detail viewing, checkout with sandbox payment processing, and booking history management.

4.2 Inputs and Validations Used

- **sessionId** — required to identify the session being booked; validated server-side via @NotNull.
- **cardholderName** — required text field; validated as non-empty on the frontend.
- **cardNumber** — pre-filled in sandbox/demo mode; numeric format enforced.
- **expiry (MM/YY)** and **CVV** — required fields with format validation.
- **JWT Bearer Token** — all protected endpoints require a valid token; missing or expired tokens return HTTP 401.
- **Capacity enforcement** — backend rejects bookings when session.bookedCount >= session.capacity (returns HTTP 422 BOOKING-001).
- **Spring Boot constraints** — @NotNull and @NotBlank validate the booking request body server-side before persistence.

4.3 How the Feature Works

1. User navigates to the **Sessions** page which fetches all available sessions via GET /api/public/sessions.
2. User clicks a session card to view the **Session Detail** page, loading full details via GET /api/public/sessions/{sessionId}.
3. User clicks **Reserve Spot** and is redirected to the **Booking Checkout** page.
4. User reviews session details and fills out the payment form (sandbox mode — Stripe test keys).
5. Frontend sends POST /api/bookings with the sessionId and paymentMethodId, including the JWT in the Authorization header.
6. Backend validates the request, checks session capacity, creates a booking record, and records the payment transaction.
7. On success, user is redirected to **My Bookings** (GET /api/me/bookings) with a success confirmation toast.

4.4 API Endpoints Used

Method	Endpoint	Purpose	Auth
GET	/api/public/sessions	List all available sessions	None
GET	/api/public/sessions/{sessionId}	Get details of a specific session	None
POST	/api/bookings	Create a new booking (with payment)	JWT Bearer
GET	/api/me/bookings	Get current user's booking history	JWT Bearer

4.5 Database Tables Involved

Table	Operation	Purpose
sessions	READ	Display available sessions and their details (title, capacity, instructor, price, thumbnail).
bookings	READ / WRITE	A new row is inserted when a user reserves a spot. Status tracked as CONFIRMED / CANCELLED.
payments	WRITE	Payment transaction record created alongside each booking (sandbox Stripe data).
users	READ	The authenticated user's ID is linked to the booking record to associate ownership.